

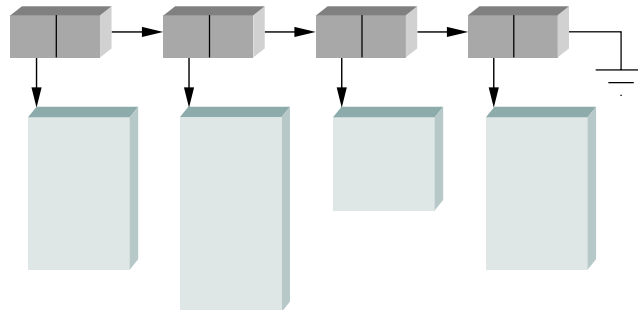


Figure 1.18 Example message data structure.

1.5 Performance



Up to this point, we have focused primarily on the functional aspects of a network. Like any computer system, however, computer networks are also expected to perform well. This is because the effectiveness of computations distributed over the network often depends directly on the efficiency with which the network delivers the computation's data. While the old programming adage “first get it right and then make it fast” is valid in many settings, in networking it is usually necessary to “design for performance.” It is, therefore, important to understand the various factors that impact network performance.

1.5.1 Bandwidth and Latency

Network performance is measured in two fundamental ways: *bandwidth* (also called *throughput*) and *latency* (also called *delay*). The bandwidth of a network is given by the number of bits that can be transmitted over the network in a certain period of time. For example, a network might have a bandwidth of 10 million bits/second (Mbps), meaning that it is able to deliver 10 million bits every second. It is sometimes useful to think of bandwidth in terms of how long it takes to transmit each bit of data. On a 10-Mbps network, for example, it takes 0.1 microsecond (μs) to transmit each bit.

While you can talk about the bandwidth of the network as a whole, sometimes you want to be more precise, focusing, for example, on the bandwidth

Bandwidth and Throughput

Bandwidth and *throughput* are two of the most confusing terms used in networking. While we could try to give you a precise definition of each term, it is important that you know how other people might use them and for you to be aware that they are often used interchangeably. First of all, bandwidth is literally a measure of the width of a frequency band. For example, a voice-grade telephone line supports a frequency band ranging from

300 to 3,300 Hz; it is said to have a bandwidth of $3,300 \text{ Hz} - 300 \text{ Hz} = 3,000 \text{ Hz}$. If you see the word “bandwidth” used in a situation in which it is being measured in hertz, then it probably refers to the range of signals that can be accommodated.

When we talk about the bandwidth of a communication link, we normally refer to the number of bits per second that can be transmitted on the link. We might say that the bandwidth of an Ethernet is 10 Mbps. A useful distinction might be made, however, between the bandwidth that is available on the link and the number of bits per second that we can actually transmit over the link in practice. We tend to use the word “throughput” to refer to the *measured performance* of a system. Thus, because of

of a single physical link or a logical process-to-process channel. At the physical level, bandwidth is constantly improving, with no end in sight. Intuitively, if you think of a second of time as a distance you could measure with a ruler, and bandwidth as how many bits fit in that distance, then you can think of each bit as a pulse of some width. For example, each bit on a 1-Mbps link is $1 \mu\text{s}$ wide, while each bit on a 2-Mbps link is $0.5 \mu\text{s}$ wide, as illustrated in Figure 1.19. The more sophisticated the transmitting and receiving technology, the narrower each bit can become, and thus, the higher the bandwidth. For logical process-to-process channels, bandwidth is also influenced by other factors, including how many times the software that implements the channel has to handle, and possibly transform, each bit of data.

The second performance metric, latency, corresponds to how long it takes a message to travel from one end of a network to the other. (As with bandwidth, we could be focused on the latency of a single link or an end-to-end channel.) Latency is measured strictly in terms of time. For example, a transcontinental network might have a latency of

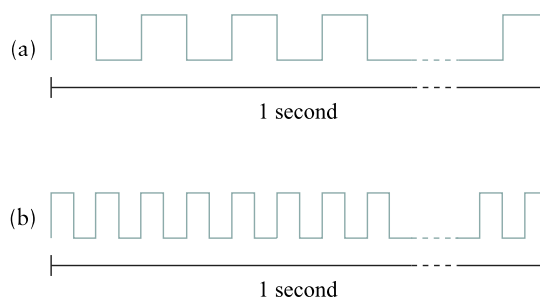


Figure 1.19 Bits transmitted at a particular bandwidth can be regarded as having some width: (a) bits transmitted at 1 Mbps (each bit $1 \mu\text{s}$ wide); (b) bits transmitted at 2 Mbps (each bit $0.5 \mu\text{s}$ wide).

24 milliseconds (ms); that is, it takes a message 24 ms to travel from one end of North America to the other. There are many situations in which it is more important to know how long it takes to send a message from one end of a network to the other and back, rather than the one-way latency. We call this the *round-trip time (RTT)* of the network.

We often think of latency as having three components. First, there is the speed-of-light propagation delay. This delay occurs because nothing, including a bit on a wire, can travel faster than the speed of light. If you know the distance between two points, you can calculate the speed-of-light latency, although you have to be careful because light travels across different mediums at different speeds: It travels at 3.0×10^8 m/s in a vacuum, 2.3×10^8 m/s in a cable, and 2.0×10^8 m/s in a fiber. Second, there is the amount of time it takes to transmit a unit of data. This is a function of the network bandwidth and the size of the packet in which the data is carried. Third, there may be queuing delays inside the network, since packet switches generally need to store packets for some time before forwarding them on an outbound link, as discussed in Section 1.2.2. So, we could define the total latency as

$$\begin{aligned}\text{Latency} &= \text{Propagation} + \text{Transmit} + \text{Queue} \\ \text{Propagation} &= \text{Distance} / \text{SpeedOfLight} \\ \text{Transmit} &= \text{Size} / \text{Bandwidth}\end{aligned}$$

where **Distance** is the length of the wire over which the data will travel, **SpeedOfLight** is the effective speed of light over that wire, **Size** is the size of the packet, and **Bandwidth** is the bandwidth at which the packet is transmitted. Note that if the message contains only one bit and we are talking about a single link (as opposed to a whole network), then the **Transmit** and **Queue** terms are not relevant, and latency corresponds to the propagation delay only.

Bandwidth and latency combine to define the performance characteristics of a given link or channel. Their relative importance, however, depends on the application.

various inefficiencies of implementation, a pair of nodes connected by a link with a bandwidth of 10 Mbps might achieve a throughput of only 2 Mbps. This would mean that an application on one host could send data to the other host at 2 Mbps.

Finally, we often talk about the bandwidth *requirements* of an application. This is the number of bits per second that it needs to transmit over the network to perform acceptably. For some applications, this might be “whatever I can get”; for others, it might be some fixed number (preferably no more than the available link bandwidth); and for others, it might be a number that varies with time. We will provide more on this topic later in this section.

For some applications, latency dominates bandwidth. For example, a client that sends a 1-byte message to a server and receives a 1-byte message in return is latency bound. Assuming that no serious computation is involved in preparing the response, the application will perform much differently on a transcontinental channel with a 100-ms RTT than it will on an across-the-room channel with a 1-ms RTT. Whether the channel is 1 Mbps or 100 Mbps is relatively insignificant, however, since the former implies that the time to transmit a byte (**Transmit**) is $8\ \mu\text{s}$ and the latter implies $\text{Transmit} = 0.08\ \mu\text{s}$.

In contrast, consider a digital library program that is being asked to fetch a 25-megabyte (MB) image—the more bandwidth that is available, the faster it will be able to return the image to the user. Here, the bandwidth of the channel dominates performance. To see this, suppose that the channel has a bandwidth of 10 Mbps. It will take 20 seconds to transmit the image, making it relatively unimportant if the image is on the other side of a 1-ms channel or a 100-ms channel; the difference between a 20.001-second response time and a 20.1-second response time is negligible.

Figure 1.20 gives you a sense of how latency or bandwidth can dominate performance in different circumstances. The graph shows how long it takes to move objects of

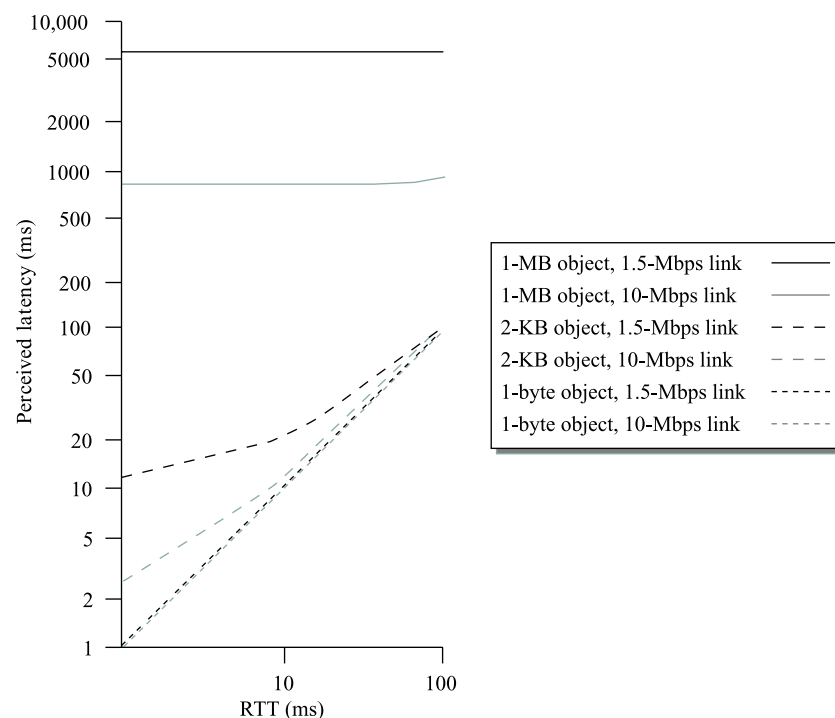


Figure 1.20 Perceived latency (response time) versus round-trip time for various object sizes and link speeds.

various sizes (1 byte, 2 KB, 1 MB) across networks with RTTs ranging from 1 to 100 ms and link speeds of either 1.5 or 10 Mbps. We use logarithmic scales to show relative performance. For a 1-byte object (say, a keystroke), latency remains almost exactly equal to the RTT, so that you cannot distinguish between a 1.5-Mbps network and a 10-Mbps network. For a 2-KB object (say, an email message), the link speed makes quite a difference on a 1-ms-RTT network but a negligible difference on a 100-ms-RTT network. And for a 1-MB object (say, a digital image), the RTT makes no difference—it is the link speed that dominates performance across the full range of RTT.

Note that throughout this book we use the terms *latency* and *delay* in a generic way, that is, to denote how long it takes to perform a particular function such as delivering a message or moving an object. When we are referring to the specific amount of time it takes a signal to propagate from one end of a link to another, we use the term *propagation delay*. Also, we make it clear in the context of the discussion whether we are referring to the one-way latency or the round-trip time.

As an aside, computers are becoming so fast that when we connect them to networks, it is sometimes useful to think, at least figuratively, in terms of *instructions per mile*. Consider what happens when a computer that is able to execute 1 billion instructions per second sends a message out on a channel with a 100-ms RTT. (To make the math easier, assume that the message covers a distance of 5,000 miles.) If that computer sits idle the full 100 ms waiting for a reply message, then it has forfeited the ability to execute 100 million instructions, or 20,000 instructions per mile. It had better have been worth going over the network to justify this waste.

1.5.2 Delay × Bandwidth Product

It is also useful to talk about the product of these two metrics, often called the *delay × bandwidth product*. Intuitively, if we think of a channel between a pair of processes as a hollow pipe (see Figure 1.21), where the latency corresponds to the length of the pipe and the bandwidth gives the diameter of the pipe, then the delay × bandwidth product gives the volume of the pipe—the maximum number of bits that could be in transit through the pipe at any given instant. Said another way, if latency (measured in time) corresponds to the length of the pipe, then given the width of each bit (also measured in time), you can calculate how many bits fit in the pipe. For example, a transcontinental channel with a one-way latency of 50 ms and a bandwidth of 45 Mbps is able to hold

$$\begin{aligned} &50 \times 10^{-3} \text{ sec} \times 45 \times 10^6 \text{ bits/sec} \\ &= 2.25 \times 10^6 \text{ bits} \end{aligned}$$

or approximately 280 KB of data. In other words, this example channel (pipe) holds as many bytes as the memory of a personal computer from the early 1980s could hold.



Figure 1.21 Network as a pipe.

How Big Is a Mega?

There are several pitfalls you need to be aware of when working with the common units of networking—MB, Mbps, KB, and Kbps. The first is to distinguish carefully between bits and bytes. Throughout this book, we always use a lowercase *b* for bits and a capital *B* for bytes. The second is to be sure you are using the appropriate definition of mega (M) and kilo (K). *Mega*, for example, can mean either 2^{20} or 10^6 . Similarly, *kilo* can be either 2^{10} or 10^3 . What is worse, in networking we typically use both definitions. Here's why.

Network bandwidth, which is often specified in terms of Mbps, is typically governed by the speed of the clock that paces the transmission of the bits. A clock that is running at 10 MHz is used to transmit bits at 10 Mbps. Because the *mega* in MHz means 10^6 hertz, Mbps is usually also defined as 10^6 bits per second. (Similarly, Kbps is 10^3 bits per second.) On the other hand, when we talk about a message that we want to transmit, we often give its size in kilobytes.

The delay $\times$ bandwidth product is important to know when constructing high-performance networks because it corresponds to how many bits the sender must transmit before the first bit arrives at the receiver. If the sender is expecting the receiver to somehow signal that bits are starting to arrive, and it takes another channel latency for this signal to propagate back to the sender (i.e., we are interested in the channel's RTT rather than just its one-way latency), then the sender can send up to two delay $\times$ bandwidths worth of data before hearing from the receiver that all is well. The bits in the pipe are said to be "in flight," which means that if the receiver tells the sender to stop transmitting, it might receive up to a delay $\times$ bandwidth's worth of data before the sender manages to respond. In our example above, that amount corresponds to 5.5×10^6 bits (671 KB) of data. On the other hand, if the sender does not fill the pipe—send a whole delay $\times$ bandwidth product's worth of data before it stops to wait for a signal—the sender will not fully utilize the network.

Note that most of the time we are interested in the RTT scenario, which we simply refer to as the delay $\times$ bandwidth product, without explicitly saying that this

Link Type	Bandwidth (Typical)	Distance (Typical)	Round-trip Delay	Delay $\times$ BW
Dial-up	56 Kbps	10 km	87 μ s	5 bits
Wireless LAN	54 Mbps	50 m	0.33 μ s	18 bits
Satellite	45 Mbps	35,000 km	230 ms	10 Mb
Cross-country fiber	10 Gbps	4,000 km	40 ms	400 Mb

Table 1.1 Sample delay $\times$ bandwidth products.

product is multiplied by two. Again, whether the “delay” in “delay $\times$ bandwidth” means one-way latency or RTT is made clear by the context. Table 1.1 shows some examples of delay $\times$ bandwidth products for some typical network links.

1.5.3 High-Speed Networks

The bandwidths available on today’s networks are increasing at a dramatic rate, and there is eternal optimism that network bandwidth will continue to improve. This causes network designers to start thinking about what happens in the limit, or stated another way, what is the impact on network design of having infinite bandwidth available.

Although high-speed networks bring a dramatic change in the bandwidth available to applications, in many respects their impact on how we think about networking comes in what does *not* change as bandwidth increases: the speed of light. To quote Scotty from *Star Trek*, “You cannae change the laws of physics.” In other words, “high speed” does not mean that latency improves at the same rate as band-

Because messages are stored in the computer’s memory, and memory is typically measured in powers of two, the K in KB is usually taken to mean 2^{10} . (Similarly, MB usually means 2^{20} .) When you put the two together, it is not uncommon to talk about sending a 32-KB message over a 10-Mbps channel, which should be interpreted to mean $32 \times 2^{10} \times 8$ bits are being transmitted at a rate of 10×10^6 bits per second. This is the interpretation we use throughout the book, unless explicitly stated otherwise.

The good news is that many times we are satisfied with a back-of-the-envelope calculation, in which case it is perfectly reasonable to pretend that a byte has 10 bits in it (making it easy to convert between bits and bytes) and that 10^6 is really equal to 2^{20} (making it easy to convert between the two definitions of mega). Notice that the first approximation introduces a 20% error, while the latter introduces only a 5% error.

To help you in your quick-and-dirty calculations, 100 ms is a reasonable number to use for a cross-country round-trip time—at least when the country in question is the United States—and 1 ms is a good approximation of an RTT across a local area network. In the case of the former, we increase the 48-ms round-trip time implied by the speed of light over a fiber to 100 ms because there are, as we have said, other sources of delay, such as the queueing time in the switches inside the network. You can also be sure that the path taken by the fiber between two points will not be a straight line.

width; the transcontinental RTT of a 1-Gbps link is the same 100 ms as it is for a 1-Mbps link.

To appreciate the significance of ever-increasing bandwidth in the face of fixed latency, consider what is required to transmit a 1-MB file over a 1-Mbps network versus over a 1-Gbps network, both of which have an RTT of 100 ms. In the case of the 1-Mbps network, it takes 80 round-trip times to transmit the file; during each RTT, 1.25% of the file is sent. In contrast, the same 1-MB file doesn't even come close to filling 1 RTT's worth of the 1-Gbps link, which has a delay $\times$ bandwidth product of 12.5 MB.

Figure 1.22 illustrates the difference between the two networks. In effect, the 1-MB file looks like a stream of data that

needs to be transmitted across a 1-Mbps network, while it looks like a single packet on a 1-Gbps network. To help drive this point home, consider that a 1-MB file is to a 1-Gbps network what a 1-KB *packet* is to a 1-Mbps network.

Another way to think about the situation is that more data can be transmitted during each RTT on a high-speed network, so much so that a single RTT becomes a significant amount of time. Thus, while you wouldn't think twice about the difference between a file transfer taking 101 RTTs rather than 100 RTTs (a relative difference of only 1%), suddenly the difference between 1 RTT and 2 RTTs is significant—a 100% increase. In other words, latency, rather than throughput, starts to dominate our thinking about network design.

Perhaps the best way to understand the relationship between throughput and latency is to return to basics. The effective end-to-end throughput that can be achieved over a network is given by the simple relationship

$$\text{Throughput} = \text{TransferSize} / \text{TransferTime}$$

where **TransferTime** includes not only the elements of one-way **Latency** identified earlier in this section, but also any additional time spent requesting or setting up the transfer. Generally, we represent this relationship as

$$\text{TransferTime} = \text{RTT} + 1/\text{Bandwidth} \times \text{TransferSize}$$

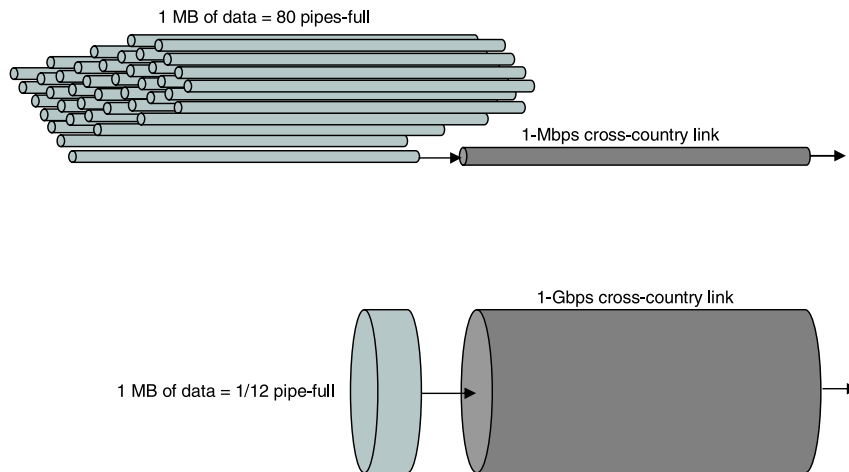


Figure 1.22 Relationship between bandwidth and latency. A 1-MB file would fill the 1-Mbps link 80 times, but only fill the 1-Gbps link 1/12 of one time.

We use **RTT** in this calculation to account for a request message being sent across the network and the data being sent back. For example, consider a situation where a user wants to fetch a 1-MB file across a 1-Gbps network with a round-trip time of 100 ms. The **TransferTime** includes both the transmit time for 1 MB ($1/1 \text{ Gbps} \times 1 \text{ MB} = 8 \text{ ms}$), and the 100-ms **RTT**, for a total transfer time of 108 ms. This means that the effective throughput will be

$$1 \text{ MB}/108 \text{ ms} = 74.1 \text{ Mbps}$$

not 1 Gbps. Clearly, transferring a larger amount of data will help improve the effective throughput, where in the limit, an infinitely large transfer size will cause the effective throughput to approach the network bandwidth. On the other hand, having to endure more than 1 **RTT**—for example, to retransmit missing packets—will hurt the effective throughput for any transfer of finite size and will be most noticeable for small transfers.

1.5.4 Application Performance Needs

The discussion in this section has taken a network-centric view of performance; that is, we have talked in terms of what a given link or channel will support. The unstated assumption has been that application programs have simple needs—they want as much bandwidth as the network can provide. This is certainly true of the aforementioned digital library program that is retrieving a 25-MB image; the more bandwidth that is available, the faster the program will be able to return the image to the user.